

SUMMER INTERNSHIP REPORT

June 2025
Second Year



Enrollment No.:
ADT23SOCB1303
Name of Student: Vibhor Dev
Class: TY-CORE-1
Department: Cse-Core

Declaration

I hereby declare that the Internship Report entitled “**Summer Internship Report**”, submitted to **MIT School of Computing, MIT Art, Design & Technology University, Pune**, is a record of the original work done by me during my internship from 6/16/2025 to 7//2025 at **DRDO**

I further declare that this report has not been submitted previously, either in part or in full, for the award of any other degree, diploma, or academic credit.

The contents of this report are based on the work performed by me under the guidance and supervision of **Mr. Pankaj Kumar** and have been duly verified by the concerned authorities.

Signature of the Student

Name : Vibhor Dev

Enrollment No. : ADT23SOCB1303

Class: TY B.Tech.: ☒ CSE ☐ IT

Date : 9/2/2025

Acknowledgement

I would like to express my sincere gratitude to **MIT School of Computing, MIT Art, Design & Technology University, Pune**, for providing me the opportunity to undertake this **2-Credit Summer Internship** as part of the academic curriculum.

I am deeply thankful to **Prof. Dr. Rajneeshkaur Sachdeo**, Dean (Academics) – MIT ADT University, **Prof. Dr. Ganesh Pathak**, Dean – MIT School of Computing, **Prof. Dr. Shraddha Phansalkar**, Associate Dean (Academics) - MIT School of Computing for their continuous encouragement and support in promoting industry exposure and experiential learning for students.

I would like to thank **Prof. Dr. Suvarna Pawar**, Head of the Department, for her constant motivation and guidance throughout the internship process.

My heartfelt thanks to my **Industry Mentor, Pankaj Kumar**, at **DRDO**, for providing meaningful work and mentorship during the internship period.

I am also thankful to **Prof. Vilas Khedekar**, Internship Nodal Officer, for his excellent coordination, timely communication, and academic mentorship during the internship program.

I extend my sincere thanks to my **Internship Coordinator**, **Prof. D. R. Kale**, and all faculty members who contributed directly or indirectly to the successful completion of this internship.

Lastly, I thank my family and friends for their continuous support and encouragement.

This internship has been a valuable learning experience, contributing significantly to my technical, professional, and personal development.

Signature of the Student

Name : Vibhor Dev

Enrollment No. : ADT23SOCB1303

Class: TY B.Tech. : ☒ CSE ☐ IT

Date : 9/2/2025

Summer Internship Completion Certificate

(For Academic Year 2025–26)

This is to certify that **Mr. Vibhor Dev**, bearing Enrollment No. **ADT23SOCB1303**, from TY B.Tech. CSE, has successfully completed the mandatory 2-Credit Summer Internship as per university guidelines.

The internship was conducted from 6/09/2025 To 7/04/2025, involving a total of 60 / 60 hours of engagement under the category:

Internship Type:

- ☐ Project-based Internship
- ☒ Training + Mini Project
- ☐ Real-world Exposure Internship (Field Work)
- ☐ NGO / CSR Tech Work
- ☐ Research Internship
- ☐ International Internship / Visit (8–10 days)

The student has submitted all required documents including the logbook, report, and presentation, and has been evaluated by the Internship Panel.

Marks Awarded: _____ / 50

Evaluation Date: ____ / ____ / 2025

Evaluation Panel Signatures:

Sr. No.	Examiner	Name of Faculty	Name of Institute	Sign. with Date
1	Faculty Mentor			
2	Internal Examiner			
3				
4				

SEAL

Internship Nodal Officer
Prof. Vilas Khedekar
MIT School of Computing

Head of Department
CSE / IT
MIT School of Computing

Prof. Dr. Shradha Phansalkar
Associate Dean (Academics)

Prof. Dr. Ganesh Pathak
Dean – School of Computing

Date: 30/05/2025

TO WHOMSOEVER IT MAY CONCERN

This is to certify that **Mr./Ms.** Vibhor Dast is a bonafide student of the **Department of Computer Science and Engineering / Information Technology** with specialization in (CSE-CORE), bearing enrollment number [ADT23SOCE1303], at School of Computing, MIT Art, Design and Technology University, Pune.

The student has been granted permission to undergo a 2 / 3 / 4-week **internship** as part of their academic curriculum at:

Company/Organization: DRDO

Address: DRDO SSPL

The internship is scheduled from 10/06/2025 to 15/07/2025 and may be carried out at the above organization in a domain relevant to the student's field of study. The primary objective of this internship is to provide **practical exposure and hands-on experience**, complementing the student's academic learning and fostering professional development.

Internship Type (Please tick one):

- ☐ Project-based Internship
- ☐ Training + Mini Project
- ☐ Real-world Exposure Internship (Field Work)
- ☐ NGO / CSR Tech Work
- ☒ Research Internship
- ☐ International Internship / Visit (8-10 days)

We wish the student all the best in their internship endeavor and look forward to their continued growth. Should you require any further information, please feel free to contact us.

Sign. Shweta

Name: SHWETA YADAV
Internship Coordinator / Nodal Officer
MIT School of Computing, Pune



2-CREDIT SUMMER INTERNSHIP FOR SY STUDENTS

EVALUATION SHEET (100 Marks)

June – July 20____

ENROLLMENT NO. : _____

NAME OF STUDENT : _____

CLASS : _____

DEPARTMENT : _____

Ind. Mentor: Evaluator 1- Industry Mentor (45 marks)

Eval Panel: Evaluator 2: Evaluation Pannel (55 Marks)

Criteria	Description	Marks Given by	Out of Marks	Obtained Marks	Total Marks
Criteria 1	Attendance & Punctuality [10 Marks]	Ind. Mentor	10	*	
Criteria 2	Learning & Skill Acquisition [30 Marks]	Ind. Mentor	15	*	
		Eval Panel	15		
Criteria 3	Project/Task Performance [30 Marks]	Ind. Mentor	15	*	
		Eval Panel	15		
Criteria 4	Report & Documentation [10 Marks]	Eval Panel	10		
Criteria 5	Mentor / Industry Feedback – [5 Marks]	Ind. Mentor	5	*	
Criteria 6	Presentation & Viva [15 Marks]	Eval Panel	15		
TOTAL=					

* Marks filled by Ind. Mentor

Evaluation Pannel:

Sr.No.	Examiner	Name of Faculty	Name of Institute	Sign. with Date
1	Faculty Mentor			
2	Internal Examiner			
3				
4				

Criteria 1: Attendance & Punctuality (10 Marks)- Marks Given by Ind Mentors - P-Poor, E-Excellent

	Criteria	Evaluation Rubrics		Marks	ST	T
1	Attendance & Punctuality (10 Marks)	1-P	Frequently absent or delayed with no valid reason.	2		
		2	Several instances of late login/ with minimal attendance.	4		
		3	Attended regularly but inconsistently punctual or communicative.	6		
		4	Present on most days with punctuality and good communication.	8		
		5-E	Fully regular, punctual, and throughout professional	10		

Criteria 2: Learning & Skill Acquisition (30 Marks) 15+15

Criteria 2.1: Learning & Skill Acquisition (15 Marks) – To be filled by Industry Mentor

	Criteria	Sub Criteria	Evaluation Rubrics		Marks	ST	T
2	Learning & Skill Acquisition (Ind. Mentor) (15 Marks)	Understanding of Tools & Techniques (5 Marks)	1-P	No familiarity or improper use of tools/tech.	1		
			2	Basic knowledge but difficulty applying it.	2		
			3	Functional knowledge with guidance.	3		
			4	Confident usage of tools with minor support.	4		
			5-E	Proficient in tools; showed independent work.	5		
		Communication & Integrity (5 Marks)	1-P	Rarely communicated and no collaboration.	1		
			2	Communicated irregularly or unclearly.	2		
			3	Communicated adequately, moderate teamwork.	3		
			4	Communicated well, contributed in team	4		
			5-E	Excellent collaboration and communication.	5		
		Daily Progress & Evidence of Learning (5 Marks)	1-P	No logbook/records or evidence of progress.	1		
			2	Incomplete or inconsistent progress evidence.	2		
			3	Average updates with visible effort.	3		
			4	Regular entries with decent quality of learning.	4		
			5-E	Consistent, insightful & well-maintained record.	5		

Criteria 2.2: Learning & Skill Acquisition (15 Marks) – To be filled by Evaluation Panel

	Criteria	Sub Criteria	Evaluation Rubrics		Marks	ST	T
2	Learning & Skill Acquisition (Ind. Mentor) (15 Marks)	Understanding of Tools & Techniques (5 Marks)	1-P	No familiarity or improper use of tools/tech.	1		
			2	Basic knowledge but difficulty applying it.	2		
			3	Functional knowledge with guidance.	3		
			4	Confident usage of tools with minor support.	4		
			5-E	Proficient in tools; showed independent work.	5		
		Communication & Integrity (5 Marks)	1-P	Rarely communicated and no collaboration.	1		
			2	Communicated irregularly or unclearly.	2		
			3	Communicated adequately, moderate teamwork.	3		
			4	Communicated well, contributed in team	4		
			5-E	Excellent collaboration and communication.	5		
		Daily Progress & Evidence of Learning (5 Marks)	1-P	No logbook/records or evidence of progress.	1		
			2	Incomplete or inconsistent progress evidence.	2		
			3	Average updates with visible effort.	3		
			4	Regular entries with decent quality of learning.	4		
			5-E	Consistent, insightful & well-maintained record.	5		

Criteria 3: Project/Task Performance (30 Marks) - P-Poor, E-Excellent

Criteria 3.1: Project/Task Performance (15 Marks)-To be filled by Industry Mentor

	Criteria	Sub Criteria	Evaluation Rubrics		Marks	ST	T
3	Project/Task Performance (15 Marks)	Quality of Work Delivered (5 Marks)	1-P	Incomplete or irrelevant output.	1		
			2	Basic effort but below expectations.	2		
			3	Functional but lacks refinement or originality.	3		
			4	Good quality with relevant outputs.	4		
			5-E	High-quality, well-documented, impactful work.	5		
		Initiative & Ownership (5 Marks)	1-P	Passive, waited for instructions.	1		
			2	Minor independent effort shown.	2		
			3	Carried out tasks as assigned.	3		
			4	Took some initiatives beyond assigned tasks.	4		
			5-E	Took ownership and exceeded expectations.	5		
		Timeliness & Problem-Solving (5 Marks)	1-P	Delays and dependence on others.	1		
			2	Completed late with errors or excuses.	2		
			3	Met most deadlines with average issue handling.	3		
			4	Timely delivery with smart troubleshooting.	4		
			5-E	Always timely and resolved problems.	5		

Criteria 3.2: Project/Task Performance (15 Marks)-To be filled by Evaluation Panel

	Criteria	Sub Criteria	Evaluation Rubrics		Marks	ST	T
3	Project/Task Performance (15 Marks)	Quality of Work Delivered (5 Marks)	1-P	Incomplete or irrelevant output.	1		
			2	Basic effort but below expectations.	2		
			3	Functional but lacks refinement or originality.	3		
			4	Good quality with relevant outputs.	4		
			5-E	High-quality, well-documented, impactful work.	5		
		Initiative & Ownership (5 Marks)	1-P	Passive, waited for instructions.	1		
			2	Minor independent effort shown.	2		
			3	Carried out tasks as assigned.	3		
			4	Took some initiatives beyond assigned tasks.	4		
			5-E	Took ownership and exceeded expectations.	5		
		Timeliness & Problem-Solving (5 Marks)	1-P	Delays and dependence on others.	1		
			2	Completed late with errors or excuses.	2		
			3	Met most deadlines with average issue handling.	3		
			4	Timely delivery with smart troubleshooting.	4		
			5-E	Always timely and resolved problems.	5		

Criteria 4: Report & Documentation (10 Marks) – To be filled by Evaluation Panel - P-Poor, E-Excellent

	Criteria	Sub Criteria	Evaluation Rubrics		Marks	ST	T
4	Report & Documentation (10 Marks)	Report Structure & Presentation (5 Marks)	1-P	Disorganized and difficult to follow.	1		
			2	Lacks structure or clarity.	2		
			3	Acceptable structure with minor issues.	3		
			4	Clear and structured presentation.	4		
			5-E	Professionally formatted, highly readable.	5		
		Technical Understanding & Reflection (5 Marks)	1-P	No insights or understanding reflected.	1		
			2	Basic explanations, limited understanding.	2		
			3	Sufficient understanding, basic reflection	3		
			4	Thoughtful reflections, solid understanding.	4		
			5-E	Deep insights and excellent technical clarity	5		

Criteria 5: Mentor / Industry Feedback (10 Marks)- To be filled by Industry Mentor

	Criteria	Sub Criteria	Evaluation Rubrics		Marks	ST	T
5	Mentor / Industry Feedback (5 Marks)		1-P	Negative feedback or lack of engagement.	1		
			2	Minimal contribution or needed push.	2		
			3	Satisfactory performance.	3		
			4	Positive and reliable engagement.	4		
			5-E	Highly appreciated and exemplary conduct.	5		

Criteria 6: Presentation & Viva (15 Marks) – To be filled by Evaluation Panel

	Criteria	Sub Criteria	Evaluation Rubrics		Marks	ST	T
6	Presentation & Viva (15 Marks)	Clarity of Communication (5 Marks)	1-P	Unclear and disorganized presentation.	1		
			2	Basic clarity, difficult to follow.	2		
			3	Acceptable flow with minor issues.	3		
			4	Clear explanation and structure.	4		
			5-E	Fluent, confident, and engaging.	5		
		Response to Questions (5 Marks)	1-P	Unable to respond or off-topic.	1		
			2	Incomplete or unclear responses.	2		
			3	Moderate accuracy in responses.	3		
			4	Clear, relevant, and timely answers.	4		
			5-E	Deep understanding, confidence.	5		
		Self-Reflection on Learning (5 Marks)	1-P	No reflection or irrelevant responses.	1		
			2	Basic effort, not connected to learning.	2		
			3	Moderate understanding of learning gains.	3		
			4	Reflective and thoughtful articulation.	4		
			5-E	Deeply insightful and growth-oriented reflections.	5		

STUDENT REPORT INDEX

Sr. No.	Topic	Sub-Topic	Page
1	Introduction		
	1.1	Objective of the Internship	2
	1.2	Overview of the Company	2
	1.3	Duration and Nature of Work	3
2	Company Profile		
	2.1	Background and History	4
	2.2	Vision, Mission, and Values	4
	2.3	Services and Products	4
	2.4	Technology Stack	4
3	Internship Activities		
	3.1	Week 1 – Orientation, Lab Tour & Understanding Objectives	5
	3.2	Week 2 – Literature Survey & Project Topic Finalization	5
	3.3	Week 3 – Project Development	5
	3.4	Week 4 – Documentation, Presentation & Review	5
4	Tools and Technologies Used		
	4.1	Programming Languages	6
	4.2	Development Tools and IDEs	6
	4.3	Project Management Tools	6
	4.4	Version Control Systems	6
5	Learning Outcomes		
	5.1	Technical Skills Gained	7
	5.2	Industry Exposure	7
	5.3	Competencies Developed (Technical/Communication/Teamwork)	7
6	Conclusion		
	6.1	Summary of Experience	8
	6.2	Future Scope and Problem Statements	8
7	Annexure / Appendix		
	7.1	Internship Completion Certificate	9
	7.2	Code Snippets and Screenshots	10
	7.3	Feedback from Industry Mentor Industry Feedback Summary	11
	7.4	Photos with geotags and Reviews	12
	7.5	Logbook	13-14

1. INTRODUCTION

1.1 Objective of the Internship

The main objective of this internship at DRDO SSPL Laboratory was to acquire hands-on experience in research-driven software development, focusing on scientific applications relevant to national defense technologies. The internship provided an opportunity to collaborate with experts in defense research to conceptualize, design, and implement a system for laboratory data automation and experimental results visualization. Key aims included:

- Applying academic knowledge to solve real-world scientific problems.
- Enhancing technical skills in Python, data handling, and analytics for laboratory automation.
- Learning structured scientific workflows and the use of specialized laboratory software tools.
- Strengthening collaboration, communication, and the ability to work in a high-security, multidisciplinary environment.

1.2 Overview of the Company

Defense Research & Development Organization (DRDO) - Solid State Physics Laboratory (SSPL), Delhi
DRDO SSPL is a premier government research facility under the Ministry of Defense, focused on materials science, semiconductor devices, and advanced electronics for national defense. SSPL specializes in developing new solid-state materials, characterizing crystal structures, and fabricating high-reliability sensors for military applications. The laboratory is at the forefront of research, providing technological solutions and innovations for various defense programs.

1.3 Duration and Nature of Work

Duration

The internship took place from 09 June 2025 to 04 July 2025, spanning four weeks and totalling more than 60 working hours. The engagement involved developing software scripts for automating laboratory experiments, processing sensor data, and creating dashboards for visualizing experimental measurements. Responsibilities included coding, data analysis, attending research meetings, and presenting project outcomes

Nature of Work

Unlike a traditional industry internship, this engagement was a structured, project-based collaboration directly with the research team at DRDO SSPL Laboratory. The internship focused on building a laboratory data automation system to streamline scientific workflows, improve data integrity, and support advanced experimental visualization.

Problem Identification & Proposal

During initial discussions with laboratory mentors and scientists, it became clear that much of the experimental data acquisition and record-keeping relied on manual processes—resulting in fragmented datasets and reduced efficiency during research cycles. To address these challenges, the proposal centred on developing a Python-based laboratory automation dashboard that would centralize experiment records, automate data logging, and facilitate better oversight of measurement results

Approach

A lightweight, web-based solution was adopted, designed to be easy to use by both scientists and technical assistants on laboratory computers and tablets. The development process combined open-source libraries (Streamlit, matplotlib, pandas) with best practices in Python scripting and scientific workflow automation. Extensive manual debugging and testing was conducted to ensure stability and reliability in a high-security, mission-critical environment.

Key Deliverables

- A secure user authentication module to restrict access to sensitive research data.
- A central dashboard displaying all ongoing experiments, real-time sensor measurements, and historical trends.
- Structured management of experiment tasks, sub-tasks, and execution methods, with assignment options for research staff.
- Integrated tracking for experiment costs, resource usage, and budget compliance.
- Automated export functionality to generate formatted reports for laboratory use and research review.

Roles & Responsibilities

- **UI/UX Design:** Designed a user-centered dashboard for ease of use by laboratory staff and researchers.
- **Python Development:** Built and refined automation scripts for data acquisition, analysis, and visualization.
- **Debugging & Refinement:** Iteratively tested the solution, resolved device connectivity issues, and optimized code for accuracy and security.
- **Deployment & Feedback Loop:** Deployed the prototype within laboratory systems; collected feedback from research mentors and made iterative improvements.

This project-driven approach simulated the full lifecycle of a research software development task—starting from problem identification and conceptualization, through coding, deployment, user acceptance, and final delivery—mirroring the collaborative and iterative nature of real-world research environments at DRDO SSPL.

2. COMPANY PROFILE

2.1 Background and History

DRDO SSPL Laboratory was established in 1962 to further India's capabilities in solid-state physics and microelectronics. Over the decades, SSPL has contributed significantly to indigenous research in crystal growth, semiconductor device fabrication, and defense sensor technologies. Led by top scientists, SSPL continues to support national strategic programs with advanced R&D facilities and expertise.

2.2 Vision, Mission, and Values

Vision: To support national defense through pioneering research in solid-state physics and electronics.

Mission: To develop robust materials and sensors for defense applications through cutting-edge laboratory research and collaboration.

Values: Commitment to innovation, scientific rigor, integrity, and confidentiality in all operations.

2.3 Services and Products

SSPL provides core research and development services in:

- Crystal growth and materials synthesis
- Design and testing of magneto resistive and photonic sensors
- Advanced microelectronics for missiles and communication systems
- Technical consulting and scientific collaboration for defense agencies

2.4 Technology Stack

SSPL utilizes laboratory instrumentation (XRD, SEM, spectrometers), specialized software for experiment automation, and programming languages such as Python and MATLAB for data processing. For development, tools like Anaconda, Spyder IDE, Git version control, and matplotlib/seaborn for data visualization were used

3. INTERNSHIP ACTIVITIES

3.1 Week 1 – Orientation, Lab Tour & Understanding Objectives

The first week involved an official orientation by DRDO SSPL Laboratory staff and a comprehensive tour of the various research labs and facilities, including crystal growth setups, clean rooms, and instrumentation halls. Interactions with scientists and mentors introduced the ongoing research themes, laboratory safety protocols, and the overall workflow followed within SSPL. The objectives and guidelines for the upcoming mini project were discussed, and project expectations were clarified.

3.2 Week 2 – Literature Survey & Project Topic Finalization

The second week focused on studying scientific literature relevant to solid state physics and microelectronics. Guidance was taken from mentors to select a feasible mini-project topic that could be completed with available resources and knowledge. The finalized topic involved designing a theoretical dashboard concept (without real lab data) for streamlining project tracking or experiment documentation within the laboratory environment. Preliminary sketches and system flow diagrams were created accordingly.

3.3 Week 3 – Project Development

During the third week, development of the mini-project was initiated. The work involved designing the user interface and data structures of the dashboard prototype using Python and basic web technologies. Dummy datasets were created to simulate experiment tracking and workflow management. The prototype focused on features such as project/task lists, status indicators, and a basic notes section. Iterative improvements were made to the interface, with feedback taken from mentors on clarity and usability.

3.4 Week 4 – Documentation, Presentation & Review

The final week was dedicated to preparing comprehensive project documentation and compiling a report on the learnings from lab observations and the mini-project. A final presentation was delivered to laboratory mentors and peers, demonstrating the key features of the conceptual dashboard and summarized learnings about SSPL research infrastructure. Feedback was received from mentors, which was then incorporated into the final report before submission and successful completion of the internship.

4. TOOLS AND TECHNOLOGIES USED\

4.1 Programming Languages

- Python: Used for hardware interfacing, data acquisition, analysis, and dashboard creation.
- MATLAB: Applied for algorithm prototyping and validation in experiment workflows.

4.2 Development Tools and IDEs

- Spyder (Anaconda): Main IDE for Python scripts.
- Jupyter Notebooks: Used for exploratory data analysis and documentation.
- Stream lit: For developing the visualization dashboard.

4.3 Project Management Tools

Task tracking was carried out via email updates and weekly review meetings, with activities documented in personal logbooks.

4.4 Version Control Systems

- Git: Managed code versions throughout the development.
- Private repository on DRDO's secure internal server.

5. LEARNING OUTCOMES

4.5 Technical Skills Gained

- Developed expertise in Python for real-time scientific data processing.
- Gained experience handling laboratory hardware via serial communication.
- Built dashboards to visualize experimental data and generate analytical reports.
- Strengthened ability to debug, optimize, and validate scientific code under supervision.

4.6 Industry Exposure

- Worked closely with defense researchers, understanding confidentiality and security requirements.
- Learned the importance of accuracy, rigorous documentation, and validation in scientific environments.
- Gained firsthand experience in transforming manual lab processes into automated, digital workflows.

4.7 Competencies Developed (Technical/Communication/Teamwork)

During the course of this internship, I was able to strengthen several important competencies that contribute to becoming industry-ready:

1. Technical Competencies

- Developed hands-on experience in Python and Streamlit for building interactive dashboards, including modular coding and UI design.
- Improved ability to troubleshoot errors, optimize logic, and document code for readability and future enhancements.
- Gained familiarity with software project structuring, version control using Git, and organizing deliverables for clarity and collaboration.

2. Communication Competencies

- Improved ability to gather requirements by discussing workflows with a non-technical mentor.
- Learned to translate technical terms into simple explanations that stakeholders could understand.
- Enhanced presentation skills by demonstrating prototypes, incorporating feedback, and documenting the project clearly.

3. Teamwork & Collaboration Competencies

- Practiced effective division of tasks and peer collaboration in mini-project planning, development, and review stages.
- Valued iterative feedback from mentors, using it to refine project scope, design, and documentation

6. CONCLUSIONS

4.8 Summary of Experience

The internship at **DRDO SSPL Laboratory** was an enriching and transformative experience that effectively bridged the gap between academic learning and its real-world application. Over the course of four weeks, I conceptualized and developed a mini-project—a laboratory dashboard prototype using Python and Streamlit—focused on simplifying research data visualization and workflow documentation within a scientific environment.

This experience helped me:

- **Gain hands-on exposure** in designing and developing a research-support tool from scratch, simulating a realistic software project lifecycle.
- **Strengthen my technical foundation** in Python programming, Streamlit-based user interface construction, version control, and academic project documentation.
- **Learn to apply programming concepts** to practical problems by translating informal laboratory workflows into structured, digital solutions.
- **Build confidence in technical communication** by presenting my work to mentors, documenting processes, and adapting features based on constructive feedback.

Overall, the internship not only enhanced my practical programming and project skills, but also provided valuable insights into the role of software in modern research organizations. It deepened my appreciation for the importance of user-centric design, clear documentation, and iterative improvement in technical projects.

4.9 Future Scope and Problem Statements

While the dashboard prototype achieved its immediate objectives, there are several promising directions for further enhancement and integration:

- **Database Integration:** Upgrading from file-based storage to a lightweight database (such as SQLite or PostgreSQL) would allow for more structured, scalable, and secure management of lab records.
- **User Authentication & Roles:** Adding secure login and differentiated access for various users (e.g., Admin, Researcher, Viewer) would boost privacy and facilitate collaboration among multiple lab members.
- **Web and Cloud Deployment:** Deploying the dashboard on an intra-lab web server or cloud platform would enable remote access, facilitating teamwork and accessibility beyond a single workstation.
- **Notification System:** Introducing automated reminders or alerts—for experiment milestones, reporting deadlines, or equipment schedules—would enhance operational efficiency.
- **Advanced Analytics & Visualization:** Integrating richer plotting and reporting options, such as dynamic charts, summary statistics, and exportable reports, would provide deeper insights to researchers.
- **Live Data Integration:** In the future, connecting the dashboard directly to laboratory instruments could automate data input and enable real-time monitoring of experiments or processes.

By addressing these opportunities, the mini-project can evolve into a full-fledged laboratory management solution, supporting more advanced research workflows, better data security, and improved collaboration within scientific teams. This iterative process reflects the continuous improvement ethos essential in both research and software development.

7. ANNEXURE / APPENDIX

```

1  def run_continuous_prediction(self, interval_hours=1):
2      # Run predictions at regular intervals
3      def job():
4          print(f"[{datetime.now()}] Running scheduled prediction...")
5          self.predict_next_price()
6
7          # Schedule job
8          schedule.every(interval_hours).hours.do(job)
9
10         print(f"Starting continuous prediction for {self.symbol}")
11         print(f"Predictions will run every {interval_hours} hour(s)")
12         print("Press Ctrl+C to stop")
13
14         # Run initial prediction
15         job()
16
17         # Keep running
18         try:
19             while True:
20                 schedule.run_pending()
21                 time.sleep(60) # Check every minute
22         except KeyboardInterrupt:
23             print("\nStopping continuous prediction...")
24
25     def create_streamlit_compatible_version(self):
26         """Generate code compatible with your Streamlit app"""
27         streamlit_code = """
28
29         import streamlit as st
30         import numpy as np
31         import pandas as pd
32         import yfinance as yf
33         from keras.models import load_model
34         import plotly.graph_objects as go
35         from sklearn.preprocessing import MinMaxScaler
36         from datetime import datetime
37
38         # Load your trained model
39         model = load_model('Stock Predictions Model.keras')
40
41         st.header('Real-time Stock Market Predictor')
42         stock = st.text_input('Enter stock Symbol', '{self.symbol}')
43
44         # Get real-time data
45         start = '2016-01-01'
46         end = datetime.now().strftime('%Y-%m-%d') # Real-time end date
47         df = yf.download(stock, start, end)
48
49         st.subheader('Stock Data')
50         st.write(df.tail(10)) # Show latest 10 days
51
52         # Chart 1: Price vs MA50 vs MA100
53         st.subheader('Price vs MA50 vs MA100')
54         ma_50_days = df.Close.rolling(50).mean()
55         ma_100_days = df.Close.rolling(100).mean()

```

```

1  def run_continuous_prediction(self, interval_hours=1):
2      def create_streamlit_compatible_version(self):
3
4          fig1 = go.Figure()
5          fig1.add_trace(go.Scatter(x=df.index, y=ma_50_days, mode='lines', name='MA50', line=dict(color='red', width=2)))
6          fig1.add_trace(go.Scatter(x=df.index, y=ma_100_days, mode='lines', name='MA100', line=dict(color='blue', width=2, dash='dash')))
7          fig1.add_trace(go.Scatter(x=df.index, y=df['Close'], mode='lines', name='Close Price', line=dict(color='green', width=2)))
8          fig1.update_layout(title='Price vs MA50 vs MA100', xaxis_title='Date', yaxis_title='Price', legend=dict(x=0, y=1, hovermode='x unified'))
9          st.plotly_chart(fig1, use_container_width=True)
10
11         # Chart 2: Price vs MA100 vs MA200
12         st.subheader('Price vs MA100 vs MA200')
13         ma_200_days = df.Close.rolling(200).mean()
14
15         fig2 = go.Figure()
16         fig2.add_trace(go.Scatter(x=df.index, y=ma_100_days, mode='lines', name='MA100', line=dict(color='red', width=2)))
17         fig2.add_trace(go.Scatter(x=df.index, y=ma_200_days, mode='lines', name='MA200', line=dict(color='blue', width=2, dash='dash')))
18         fig2.add_trace(go.Scatter(x=df.index, y=df['Close'], mode='lines', name='Close Price', line=dict(color='green', width=2)))
19         fig2.update_layout(title='Price vs MA100 vs MA200', xaxis_title='Date', yaxis_title='Price', legend=dict(x=0, y=1, hovermode='x unified'))
20         st.plotly_chart(fig2, use_container_width=True)
21
22         # Prediction Chart
23         df_train = df[['Close']][:int(len(df)*0.80)]
24         df_test = df[['Close']] [int(len(df)*0.80):]
25         scaler = MinMaxScaler(feature_range=(0,1))
26         past_100_days = df_train.tail(100)
27         df_test = pd.concat([past_100_days, df_test], ignore_index=True)
28         df_test_scaler = scaler.fit_transform(df_test)
29
30         x, y = [], []
31         for i in range(100, df_test_scaler.shape[0]):
32             x.append(df_test_scaler[i-100:i])
33             y.append(df_test_scaler[i,0])
34
35         x, y = np.array(x), np.array(y)
36         predict = model.predict(x)
37         scale = 1/scaler.scale_
38         predict = predict * scale
39         y = y * scale
40
41         st.subheader('Original Price vs Predicted Price')
42         time_index = df.index[-len(y):]
43
44         fig3 = go.Figure()
45         fig3.add_trace(go.Scatter(x=time_index, y=y, mode='lines-markers', name='Original Price', line=dict(color='red', width=2)))
46         fig3.add_trace(go.Scatter(x=time_index, y=predict.flatten(), mode='lines-markers', name='Predicted Price', line=dict(color='blue', width=2, dash='dot')))
47         fig3.update_layout(title='Original Price vs Predicted Price', xaxis_title='Date', yaxis_title='Price', legend=dict(x=0, y=1, hovermode='x unified'))
48         st.plotly_chart(fig3, use_container_width=True)
49
50         # Real-time prediction for next day
51         latest_100_days = df[['Close']].tail(100)
52         latest_100_scaled = scaler.transform(latest_100_days)
53         X_pred = np.array(latest_100_scaled)
54         next_day_prediction = model.predict(X_pred)
55         next_day_price = scaler.inverse_transform(next_day_prediction)[0][0]

```

```

1  def run_continuous_prediction(self, interval_hours=1):
2      def create_streamlit_compatible_version(self):
3
4          st.subheader('Next Day Prediction')
5          current_price = df['Close'].iloc[-1]
6          st.metric(
7              label="Tomorrow's Predicted Price",
8              value=f"${next_day_price:2f}",
9              delta=f"${(next_day_price - current_price:2f)}" / (((next_day_price - current_price) / current_price) * 100:2f)}%"
10         )
11
12         # Real-Time Stock Price Prediction System
13         # pip install alpha_vantage (already installed)
14         # pip install --upgrade tensorflow (already installed)
15         # pip install schedule (new - for automated predictions)
16         # pip install plotly (new - for interactive graphs)
17
18         import numpy as np
19         import pandas as pd
20         from pandas_datareader import data as pdr
21         import matplotlib.pyplot as plt
22         import yfinance as yf
23         import seaborn as sns
24         from sklearn.preprocessing import MinMaxScaler
25         from tensorflow.keras.layers import Dense, Dropout, LSTM
26         from tensorflow.keras.models import Sequential, load_model
27         import plotly.graph_objects as go
28         import plotly.express as px
29         from plotly.subplots import make_subplots
30         import schedule
31         import time
32         from datetime import datetime, timedelta
33         import warnings
34         warnings.filterwarnings('ignore')
35
36         class RealTimeStockPredictor:
37             def __init__(self, symbol='TSLA', model_path='Stock Predictions Model.keras'):
38                 self.symbol = symbol
39                 self.model_path = model_path
40                 self.scaler = MinMaxScaler(feature_range=(0,1))
41                 self.model = None
42                 self.sequence_length = 100 # Same as your original 100-day lookahead
43
44             def fetch_real_time_data(self, start_date='2016-01-01'):
45                 """Fetch the most recent data including today"""
46                 try:
47                     # Get data up to today
48                     end_date = datetime.now().strftime('%Y-%m-%d')
49                     print(f"Fetching data from {start_date} to {end_date}")
50
51                     df = yf.download(self.symbol, start=start_date, end=end_date)
52                     df.reset_index(inplace=True)

```

```

1  def run_continuous_prediction(self, interval_hours=1):
2      def create_streamlit_compatible_version(self):
3
4          class RealTimeStockPredictor:
5
6              def fetch_real_time_data(self, start_date='2016-01-01'):
7                  """Add the same technical indicators as your original code"""
8                  ma_50_days = df['Close'].rolling(window=100).mean()
9                  ma_150_days = df['Close'].rolling(window=150).mean()
10
11                  # Store for plotting if needed
12                  self.ma_50 = ma_50_days
13                  self.ma_150 = ma_150_days
14
15                  df.dropna(inplace=True)
16                  print(f"Data fetched successfully. Latest date: {df['Date'].iloc[-1]}")
17                  return df
18
19              except Exception as e:
20                  print(f"Error fetching data: {e}")
21                  return None
22
23              def prepare_data_for_training(self, df):
24                  """Prepare data the same way as your original code"""
25                  # Split data (80% train, 20% test) - but now with real-time data
26                  split_point = int(len(df) * 0.80)
27                  df_train = df[['Close']][:split_point]
28                  df_test = df[['Close']] [split_point:]
29
30                  # Scale the data
31                  df_train_scale = self.scaler.fit_transform(df_train)
32                  df_test_scale = self.scaler.transform(df_test)
33
34                  return df_train, df_test, df_train_scale, df_test_scale
35
36              def create_sequences(self, data, sequence_length=100):
37                  """Create sequences for LSTM training - same as your original logic"""
38                  x, y = [], []
39                  for i in range(sequence_length, len(data)):
40                      x.append(data[i-sequence_length:i])
41                      y.append(data[i, 0])
42                  return np.array(x), np.array(y)
43
44              def build_model(self, input_shape):
45                  """Build the exact same model architecture as your original"""
46                  model = Sequential()
47
48                  # Same architecture as your original model
49                  model.add(LSTM(units=50, activation='relu', return_sequences=True, input_shape=input_shape))
50                  model.add(LSTM(units=50, activation='relu', return_sequences=True))
51                  model.add(Dropout(0.2))
52
53                  model.add(LSTM(units=60, activation='relu', return_sequences=True))
54                  model.add(Dropout(0.3))
55
56                  model.add(LSTM(units=80, activation='relu', return_sequences=True))
57                  model.add(Dropout(0.4))

```

```

spy > run_continuous_prediction > create_streamlit_compatible_version > RealTimeStockPredictor > build_model
1 def run_continuous_prediction(self, interval_hours=1):
25 def create_streamlit_compatible_version(self):
144 class RealTimeStockPredictor:
199 def build_model(self, input_shape):
213
214     model.add(LSTM(units=120, activation='relu'))
215     model.add(Dropout(0.5))
216
217     model.add(Dense(units=1))
218
219     model.compile(optimizer='adam', loss='mean_squared_error')
220     return model
221
222 def train_model(self, retrain=False):
223     """Train model with current data or load existing model"""
224     try:
225         if not retrain and self.model_path:
226             print("Loading existing model...")
227             self.model = load_model(self.model_path)
228             print("Model loaded successfully!")
229             return True
230     except:
231         print("Could not load existing model. Training new model...")
232         retrain = True
233
234     if retrain:
235         print("Fetching data for training...")
236         df = self.fetch_real_time_data()
237         if df is None:
238             return False
239
240         df_train, df_test, df_train_scale, df_test_scale = self.prepare_data_for_training(df)
241
242         # Create sequences
243         x_train, y_train = self.create_sequences(df_train_scale, self.sequence_length)
244
245         print(f"Training data shape: x={x_train.shape}, y={y_train.shape}")
246
247         # Build and train model
248         self.model = self.build_model((x_train.shape[1], 1))
249
250         print("Training model...")
251         history = self.model.fit(x_train, y_train, epochs=50, batch_size=32, verbose=1)
252
253         # Save model
254         self.model.save(self.model_path)
255         print(f"Model saved as {self.model_path}")
256
257         return True
258
259 def predict_next_price(self, plot_results=True):
260     """Make real-time prediction for next trading day"""
261     if self.model is None:
262         print("Model not loaded. Please train first.")
263         return None

```

```

#df
# Chart 1: Price vs MA50 vs MA100
144 class RealTimeStockPredictor:
145     def predict_next_price(self, plot_results=True):
254
255         # Fetch latest data
256         df = self.fetch_real_time_data()
257         if df is None:
258             return None
259
260         # Get last 100 days for prediction
261         last_100_days = df[['Close']].tail(self.sequence_length)
262
263         # Scale the data using the same scaler
264         last_100_scaled = self.scaler.transform(last_100_days)
265
266         # Reshape for prediction
267         X_pred = np.array([last_100_scaled])
268
269         # Make prediction
270         predicted_scaled = self.model.predict(X_pred, verbose=0)
271
272         # Inverse transform to get actual price
273         predicted_price = self.scaler.inverse_transform(predicted_scaled)[0][0]
274
275         current_price = df['Close'].iloc[-1]
276         prediction_date = datetime.now() + timedelta(days=1)
277
278         print("\n\n")
279         print(f"REAL-TIME STOCK PREDICTION FOR {self.symbol}")
280         print(f"\n\n")
281         print(f"Current Date: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}")
282         print(f"Latest Price: ${current_price:.2f}")
283         print(f"Predicted Price for {prediction_date.strftime('%Y-%m-%d')}: ${predicted_price:.2f}")
284         print(f"Predicted Change: ${predicted_price - current_price:.2f} ({(predicted_price - current_price) / current_price * 100:.2f}%)")
285         print(f"\n\n")
286
287     if plot_results:
288         # Create all three interactive charts
289         self.plot_interactive_charts(df)
290
291         # Create prediction comparison chart
292         self.plot_prediction_results(df)
293
294     return {
295         'current_price': current_price,
296         'predicted_price': predicted_price,
297         'prediction_date': prediction_date.strftime('%Y-%m-%d'),
298         'change': predicted_price - current_price,
299         'change_percent': ((predicted_price - current_price) / current_price) * 100
300     }
301
302 def plot_interactive_charts(self, df):
303     """Create all three interactive charts like your Streamlit app"""
304
305     # Calculate moving averages
306     ma_50_days = df['Close'].rolling(50).mean()
307     ma_100_days = df['Close'].rolling(100).mean()
308     ma_200_days = df['Close'].rolling(200).mean()
309
310     # Chart 1: Price vs MA50 vs MA100
311     fig1 = go.Figure()
312     fig1.add_trace(go.Scatter(
313         x=df['Date'], y=ma_50_days,
314         mode='lines',
315         name='MA50',
316         line=dict(color='red', width=2)
317     ))
318     fig1.add_trace(go.Scatter(
319         x=df['Date'], y=ma_100_days,
320         mode='lines',
321         name='MA100',
322         line=dict(color='blue', width=2, dash='dash')
323     ))
324     fig1.add_trace(go.Scatter(
325         x=df['Date'], y=df['Close'],
326         mode='lines',
327         name='Close Price',
328         line=dict(color='green', width=2)
329     ))
330     fig1.update_layout(
331         title=f'{self.symbol} - Price vs MA50 vs MA100',
332         xaxis_title='Date',
333         yaxis_title='Price',
334         legend=dict(x=0, y=1),
335         hovermode='x unified',
336         width=1000,
337         height=600
338     )
339     fig1.show()
340
341     # Chart 2: Price vs MA100 vs MA200
342     fig2 = go.Figure()
343     fig2.add_trace(go.Scatter(
344         x=df['Date'], y=ma_100_days,
345         mode='lines',
346         name='MA100',
347         line=dict(color='red', width=2)
348     ))
349     fig2.add_trace(go.Scatter(
350         x=df['Date'], y=ma_200_days,
351         mode='lines',
352         name='MA200',
353         line=dict(color='blue', width=2, dash='dash')
354     ))

```

```

spy > run_continuous_prediction > create_streamlit_compatible_version > RealTimeStockPredictor > plot_interactive_charts
1 def run_continuous_prediction(self, interval_hours=1):
25 def create_streamlit_compatible_version(self):
144 class RealTimeStockPredictor:
312 def plot_interactive_charts(self, df):
314
315     # Calculate moving averages
316     ma_50_days = df['Close'].rolling(50).mean()
317     ma_100_days = df['Close'].rolling(100).mean()
318     ma_200_days = df['Close'].rolling(200).mean()
319
320     # Chart 1: Price vs MA50 vs MA100
321     fig1 = go.Figure()
322     fig1.add_trace(go.Scatter(
323         x=df['Date'], y=ma_50_days,
324         mode='lines',
325         name='MA50',
326         line=dict(color='red', width=2)
327     ))
328     fig1.add_trace(go.Scatter(
329         x=df['Date'], y=ma_100_days,
330         mode='lines',
331         name='MA100',
332         line=dict(color='blue', width=2, dash='dash')
333     ))
334     fig1.add_trace(go.Scatter(
335         x=df['Date'], y=df['Close'],
336         mode='lines',
337         name='Close Price',
338         line=dict(color='green', width=2)
339     ))
340     fig1.update_layout(
341         title=f'{self.symbol} - Price vs MA50 vs MA100',
342         xaxis_title='Date',
343         yaxis_title='Price',
344         legend=dict(x=0, y=1),
345         hovermode='x unified',
346         width=1000,
347         height=600
348     )
349     fig1.show()
350
351     # Chart 2: Price vs MA100 vs MA200
352     fig2 = go.Figure()
353     fig2.add_trace(go.Scatter(
354         x=df['Date'], y=ma_100_days,
355         mode='lines',
356         name='MA100',
357         line=dict(color='red', width=2)
358     ))
359     fig2.add_trace(go.Scatter(
360         x=df['Date'], y=ma_200_days,
361         mode='lines',
362         name='MA200',
363         line=dict(color='blue', width=2, dash='dash')
364     ))

```

```

spy > run_continuous_prediction > create_streamlit_compatible_version > RealTimeStockPredictor > plot_interactive_charts
1 def run_continuous_prediction(self, interval_hours=1):
25 def create_streamlit_compatible_version(self):
144 class RealTimeStockPredictor:
312 def plot_interactive_charts(self, df):
365
366     fig2.add_trace(go.Scatter(
367         x=df['Date'], y=df['Close'],
368         mode='lines',
369         name='Close Price',
370         line=dict(color='green', width=2)
371     ))
372     fig2.update_layout(
373         title=f'{self.symbol} - Price vs MA100 vs MA200',
374         xaxis_title='Date',
375         yaxis_title='Price',
376         legend=dict(x=0, y=1),
377         hovermode='x unified',
378         width=1000,
379         height=600
380     )
381     fig2.show()
382
383     return fig1, fig2
384
385 def plot_prediction_results(self, df):
386     """Create the prediction comparison chart like your Streamlit app"""
387     # Prepare data exactly like your original code
388     df_train = df[['Close']][0:int(len(df)*0.80)]
389     df_test = df[['Close']][int(len(df)*0.80):]
390
391     past_100_days = df_train.tail(100)
392     df_test_combined = pd.concat([past_100_days, df_test], ignore_index=True)
393
394     # Scale the test data
395     scaler = MinMaxScaler(feature_range=(0,1))
396     df_test_scale = scaler.fit_transform(df_test_combined)
397
398     # Create sequences for prediction
399     x, y = [], []
400     for i in range(100, df_test_scale.shape[0]):
401         x.append(df_test_scale[i-100:i])
402         y.append(df_test_scale[i,0])
403
404     x, y = np.array(x), np.array(y)
405
406     # Make predictions
407     predict = self.model.predict(x)
408
409     # Inverse transform
410     scale = 1/scaler.scale_
411     predict = predict * scale
412     y = y * scale
413
414     # Create time index for the test period
415     test_start_idx = int(len(df) * 0.80)
416     time_index = df['Date'].iloc[test_start_idx + 100:] # Skip first 100 days used for sequences

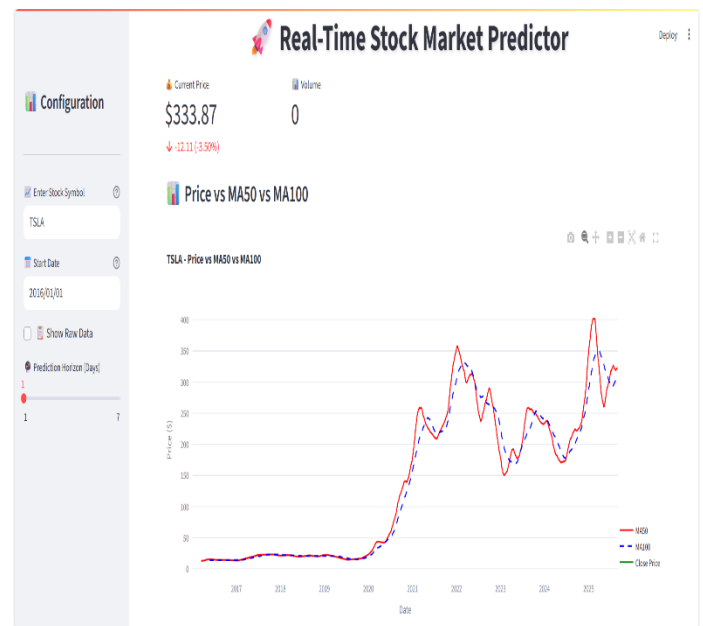
```

SUMMER INTERNSHIP REPORT, MIT SCHOOL OF COMPUTING, MIT ADT UNIVERSITY, PUNE

```
st.py > run_continuous_prediction > create_streamlit_compatible_version > RealTimeStockPredictor > plot_prediction_results
1 def run_continuous_prediction(self, interval_hours=1):
2     def create_streamlit_compatible_version(self):
144         class RealTimeStockPredictor:
384             def plot_prediction_results(self, df):
417                 # Ensure lengths match
418                 min_len = min(len(time_index), len(y), len(predict))
419                 time_index = time_index[:min_len]
420                 y = y[:min_len]
421                 predict = predict[:min_len]
422
423                 # Create the interactive prediction comparison chart
424                 fig3 = go.Figure()
425                 fig3.add_trace(go.Scatter(
426                     x=time_index, y=y,
427                     mode='lines+markers',
428                     name='Original Price',
429                     line=dict(color='red', width=2),
430                     marker=dict(size=4)
431                 ))
432                 fig3.add_trace(go.Scatter(
433                     x=time_index, y=predict.flatten(),
434                     mode='lines+markers',
435                     name='Predicted Price',
436                     line=dict(color='blue', width=2, dash='dot'),
437                     marker=dict(size=4)
438                 ))
439
440                 # Add hover information showing exact values
441                 fig3.update_traces(
442                     hovertemplate='<b>%(fullData.name)</b><br>' +
443                     'Date: %(x)<br>' +
444                     'Price: $%(y:.2f)<br>' +
445                     '<extra></extra>'
446                 )
447
448                 fig3.update_layout(
449                     title='(self.symbol) - Original Price vs Predicted Price',
450                     xaxis_title='Date',
451                     yaxis_title='Price',
452                     legend=dict(x=0, y=1),
453                     hovermode='x unified',
454                     width=1000,
455                     height=600
456                 )
457                 fig3.show()
458
459                 # Calculate and display accuracy metrics
460                 mse = np.mean((y - predict.flatten())**2)
461                 mae = np.mean(np.abs(y - predict.flatten()))
462                 mape = np.mean(np.abs(y - predict.flatten()) / y) * 100
463
464                 print(f"\nPrediction Accuracy Metrics:")
465                 print(f"Mean Squared Error (MSE): {mse:.2f}")
466                 print(f"Mean Absolute Error (MAE): {mae:.2f}")
467                 print(f"Mean Absolute Percentage Error (MAPE): {mape:.2f}%")

st.py > run_continuous_prediction > create_streamlit_compatible_version > RealTimeStockPredictor > plot_prediction_results
1 def run_continuous_prediction(self, interval_hours=1):
2     def create_streamlit_compatible_version(self):
144         class RealTimeStockPredictor:
384             def plot_prediction_results(self, df):
469                 return fig3
470
471             def create_comprehensive_analysis(self):
472                 """Create a comprehensive analysis with all charts and predictions"""
473                 print(f"Creating comprehensive analysis for {self.symbol}...")
474
475                 # Fetch latest data
476                 df = self.fetch_real_time_data()
477                 if df is None:
478                     return None
479
480                 # Create all interactive charts
481                 print("Generating interactive charts...")
482                 fig1, fig2 = self.plot_interactive_charts(df)
483
484                 # Create prediction analysis
485                 print("Generating prediction analysis...")
486                 fig3 = self.plot_prediction_results(df)
487
488                 # Make real-time prediction
489                 print("Making real-time prediction...")
490                 prediction_result = self.predict_next_price(plot_results=False)
491
492                 return {
493                     'charts': [fig1, fig2, fig3],
494                     'prediction': prediction_result,
495                     'data': df
496                 }
497                 """Run predictions at regular intervals"""
498
499             def job():
500                 print(f"\n[{datetime.now()}] Running scheduled prediction...")
501                 self.predict_next_price()
502
503                 # Schedule job
504                 schedule.every(interval_hours).hours.do(job)
505
506                 print(f"Starting continuous prediction for {self.symbol}")
507                 print(f"Predictions will run every {interval_hours} hour(s)")
508                 print("Press Ctrl+C to stop")
509
510                 # Run initial prediction
511                 job()
512
513                 # Keep running
514                 try:
515                     while True:
516                         schedule.run_pending()
517                         time.sleep(60) # Check every minute
518                 except KeyboardInterrupt:
519                     print("\nStopping continuous prediction...")
```

```
st.py > run_continuous_prediction > create_streamlit_compatible_version > RealTimeStockPredictor > plot_prediction_results
1 def run_continuous_prediction(self, interval_hours=1):
2     def create_streamlit_compatible_version(self):
144         print("\nStopping continuous prediction...")
145
520         # Example usage
521         if __name__ == "__main__":
522             # Initialize predictor
523             predictor = RealTimeStockPredictor(symbol='TSLA')
524
525             # Option 1: Train new model with latest data
526             print("Training/Loading model...")
527             if predictor.train_model(retrain=False): # Set to True to retrain with latest data
528                 print("Model trained successfully")
529
530             # Option 2: Create comprehensive analysis with all 3 charts
531             print("\nCreating comprehensive analysis...")
532             analysis = predictor.create_comprehensive_analysis()
533
534             # Option 3: Make single prediction (uncomment to use)
535             # result = predictor.predict_next_price()
536
537             # Option 4: Generate Streamlit-compatible code (uncomment to use)
538             # streamlit_code = predictor.create_streamlit_compatible_version()
539             # print("Streamlit code generated")
540
541             # Option 5: Run continuous predictions (uncomment to use)
542             # predictor.run_continuous_prediction(interval_hours=1)
543
544             print("\nReal-time prediction system ready!")
545
546             # You can also manually call predictions anytime:
547             # result = predictor.predict_next_price()
548
549             # Additional utility functions for extended functionality
550             def compare_multiple_stocks(symbols=['TSLA', 'AAPL', 'GOOGL']):
551                 """Compare predictions for multiple stocks"""
552                 predictions = {}
553
554                 for symbol in symbols:
555                     print(f"\nAnalyzing {symbol}...")
556                     predictor = RealTimeStockPredictor(symbol=symbol)
557                     if predictor.train_model(retrain=False):
558                         result = predictor.predict_next_price(plot_results=False)
559                         predictions[symbol] = result
560
561                 # Summary
562                 print(f"\n{'-'*40}")
563                 print("MULTI-STOCK PREDICTION SUMMARY")
564                 print(f"{'-'*40}")
565                 for symbol, pred in predictions.items():
566                     if pred:
567                         print(f"{symbol}: ${pred['current_price']:.2f} ~ ${pred['predicted_price']:.2f} ({pred['change_percent']:.2f}%)")
568
569                 return predictions
570
571             return predictions
572
```



Configuration

Enter Stock Symbol

TSLA

Start Date

2016/01/01

Show Raw Data

Prediction Horizon (Days)

1

7



Configuration

Enter Stock Symbol

TSLA

Start Date

2016/01/01

Show Raw Data

Prediction Horizon (Days)

1

7

Real-Time Predictions

Deploy

Tomorrow's Prediction

\$327.35

\$-6.52 (-1.95%)

Prediction for 2025-09-02

Last updated: 2025-09-01 20:24:00

Data provided by Yahoo Finance | Model: LSTM Neural Network

\$-6.52 (-1.95%)

Prediction for 2025-09-02

Extended Predictions

	Day	Date	Predicted Price	Change	Change %
0	Day +1	2025-09-02	\$327.35	\$-6.52	-1.95%
1	Day +2	2025-09-03	\$327.35	\$-6.52	-1.95%
2	Day +3	2025-09-04	\$327.35	\$-6.52	-1.95%
3	Day +4	2025-09-05	\$327.35	\$-6.52	-1.95%
4	Day +5	2025-09-06	\$327.35	\$-6.52	-1.95%

Last updated: 2025-09-01 20:28:13

Data provided by Yahoo Finance | Model: LSTM Neural Network

7.5 LOG BOOK

I hereby declare that the information recorded above is true and accurately reflects the work done by me during the internship.

Date	Time Spent (Hrs)	Activities / Tasks Completed	Tools / Platforms Used
09/06/2025	3	Orientation, introduction to lab and mentors, overview of project goals	In-person, Google Meet
10/06/2025	3	Lab tour, understanding available facilities and safety protocols	Lab visit
11/06/2025	3	Discussion on dashboard mini-project objectives, requirements finalized	Mentor discussion
12/06/2025	3	Literature review on digital dashboards and lab management	Online articles, PDF
13/06/2025	3	Prepared initial UI design and project structure sketches on paper	Pen & Paper
16/06/2025	3	Setup Python/Streamlit development environment, folder structure created	VS Code, Python, Streamlit
17/06/2025	3	Coded basic dashboard layout with placeholder sections	Streamlit, Python
18/06/2025	3	Added project/task logging section to dashboard	Streamlit, Python
19/06/2025	3	Implemented simple data entry forms, dummy data handling	Streamlit, Python
20/06/2025	3	Enhanced UI design, reviewed with peer/mentor	Streamlit, VS Code
23/06/2025	3	Incorporated feedback, included data visualization elements	Streamlit, matplotlib
24/06/2025	3	Documentation on workflow, updated in report file	MS Word
25/06/2025	3	Finalized and tested data entry and visualization modules	Streamlit, Python
26/06/2025	3	Peer code review, feedback implemented for readability and enhancements	VS Code
27/06/2025	3	Conducted usability testing, fixed minor bugs	Stream lit, Python
30/06/2025	3	Took screenshots and prepared user guide sections for the report	Snipping Tool, MS Word
01/07/2025	3	Prepared and delivered a mini-project presentation to mentor	PowerPoint, Google Meet
02/07/2025	3	Final report writing, merged project files, and review	MS Word, VS Code
03/07/2025	3	Submitted complete project package and code to mentor for evaluation	GitHub, Email
04/07/2025	3	Wrap-up, mentor feedback session, internship review and closure	In-person, Google Meet

I hereby declare that the information recorded above is true and accurately reflects the work done by me during the internship.

Student Signature: _____

Date: 2 / 9 / 2025

Mentor Feedback:

Verified by Mentor:

Name: Pankaj Kumar,

Signature: _____



ISO 9001:2015 Certified

होमावस्था भौतिकी प्रयोगशाला, दिल्ली
Solid State Physics Laboratory, Delhi



(Min. of Defence, DRDO)
Lucknow Road, Timarpur, Delhi-110054

File No. 1805... 756 .../HR/SSPL/2025
Dated ... 04-07-2025

प्रशिक्षण प्रमाणपत्र / TRAINING CERTIFICATE

This is to certify that Mr./Ms./ Vibhor Dera

Student of MIT ADT University

Roll No. ADT23 SOC B1303

Branch B.Tech CSE

has completed successfully Summer/Winter Internship for

the period from 09/06/2025 to 04/07/2025 Duration 04 weeks / months.

Topic of Internship was Real Time - Stock Price Prediction using LSTM

During the training period his/her conduct at SSPL was good.

डॉ. राम आशीष चौकसे / Dr. Ram Ashish Chouksey

वैज्ञानिक 'एफ' / Scientist 'F'

होमावस्था भौतिकी प्रयोगशाला/Solid State Physic Lab
आर. आर. रोड, रक्षा मंत्रालय/Govt. of India, Ministry of Defence
सख्तपुर मार्ग, तिमरपुर, दिल्ली Head-HR Delhi-54

MIT School of Computing



MIT-ADT
UNIVERSITY
PUNE, INDIA
A step towards World Class Education

4 Week Internship

Date 08/07/2025

INDUSTRY FEEDBACK FORM

Industry: DRDO SSPL
Industry Mentor Name: Lovely Gupta Pankaj Kumar
Mobile No 9540739137 email ID pankajkumar.sspl@govt.in
Name of Intern(s) Vibhor Desai Section/Department CSE-SY
Task/Project Assigned Real-Time Stock Price Prediction
Kindly mark 1) Excellent 2) Very Good 3) Good 4) Satisfactory 5) Poor

1. Interns are regular, hardworking and sincere during internship.
2. Interns are using advanced technology and taking interest in daily activities.
3. Interns are having good communication, technical and logical skill required for given task.
4. Interns are following instructions and fulfilling their responsibilities assigned.
5. Interns are working properly in group.
6. Interns are following industry rules and regulations as well as safety norms.
7. Interns behavior and discipline among group and with industry employees is healthy.
8. Interns are suitable for placement in same industry.

Remarks

He is good software developer in web
frameworks

Place peew

Date 08/07/25

Signature

Pankaj Kumar

PAKSHI KUMAR
SC-1E



The New Age **University** for
INNOVATION & ENTREPRENEURSHIP



MIT Art, Design & Technology University, Pune

Vishwarajbaug, Loni Kalbhor, Pune – 412 201, Maharashtra, India

Contact : 90210 80155 | www.mituniversity.ac.in

Scan and follow us to know more about MIT-ADT University, Pune

